WhatYouWear

A Full-Stack E-Commerce Web Application

React 19 · Django REST Framework · PostgreSQL · Razorpay

Project Presentation | 2025

This document contains a comprehensive overview of the WhatYouWear e-commerce project — covering its introduction, objectives, technology stack, system architecture, key features, workflow, challenges, and future scope.

■■ Section 01

Introduction

WhatYouWear is a modern, production-ready full-stack e-commerce web application built for online clothing and fashion shopping. It delivers a seamless shopping experience — from product discovery all the way through to secure checkout — targeting fashion-conscious users who demand a fast, intuitive, and mobile-friendly online store.

- Fully decoupled frontend (React) and backend (Django REST Framework)
- JWT-based authentication with Google OAuth 2.0 social login support
- Real-time payment processing via the Razorpay gateway
- Responsive UI built with Tailwind CSS and smooth Framer Motion animations
- Deployed on Vercel (frontend) with a Gunicorn/WhiteNoise backend

Section 02

Problem Statement

- Traditional e-commerce platforms are often slow, complex, and hard to navigate
- Lack of flexible Indian payment options (UPI, wallets, net banking) in many open-source demos
- No support for Google-based quick sign-in, increasing registration friction
- Difficulty managing orders, tracking status, and processing refunds transparently
- Poor mobile responsiveness and sluggish page transitions in legacy implementations
- Monolithic architectures that are hard to scale or independently deploy

Section 03

Objectives

#	Objective	Description
1	Product Catalogue	Browse, search, and filter clothing products by category
2	User Authentication	Email/password registration + Google OAuth 2.0 social login
3	Shopping Cart	Add items with colour & size selection; persist for guests
4	Secure Checkout	Shipping form + Razorpay payment gateway integration
5	Order Management	Full order lifecycle: Pending → Processing → Shipped → Delivered
6	Product Reviews	Authenticated users can rate (1–5 ★) and review products
7	Refund Tracking	Request, track, and record refunds with admin notes
8	Deployment	Frontend on Vercel; backend via Gunicorn + WhiteNoise

Section 04

Technology Stack

Layer	Technology / Library	Purpose
Frontend Framework	React 19 + Vite 7	Component-based SPA with fast HMR
Styling	Tailwind CSS 4	Utility-first responsive styling
Routing	React Router DOM v7	Client-side navigation & protected routes
Animations	Framer Motion	Page & component transition effects
HTTP Client	Axios	REST API calls with interceptors
Notifications	React Hot Toast	In-app toast alerts
Icons	Lucide React	Consistent SVG icon set
Backend Framework	Django 5 + DRF	REST API & business logic
Authentication	SimpleJWT + google-auth	JWT tokens & Google OAuth 2.0
Payment	Razorpay	Indian payment gateway
Database	PostgreSQL + psycopg2	Relational data store
ORM / Config	dj-database-url + dotenv	DB URL parsing & env management
Static / Media	WhiteNoise + Pillow	Static serving & image processing
Deployment	Vercel + Gunicorn	Frontend CDN & WSGI server

Section 05

System Architecture

WhatYouWear uses a **decoupled (headless) architecture** where the React frontend and Django backend communicate exclusively through a RESTful JSON API. This allows each tier to be developed, tested, and deployed independently.

Layer	Component	Notes
Browser / Client	React SPA (Vite build)	Served from Vercel CDN
API Transport	HTTPS + Axios	JWT Bearer token on every request
Backend API	Django REST Framework	Gunicorn WSGI server
Business Logic	Django Views / Serializers	Validation, permissions, Razorpay
Data Store	PostgreSQL	Hosted managed database
External Services	Razorpay · Google OAuth · Email	Payment, social login, password reset
Media / Static	WhiteNoise + Pillow	Product images served inline

Section 06

Key Features

User Authentication

- Email/password register, login, logout
- Google OAuth 2.0 single sign-on
- Forgot password via secure email link (24 h expiry)
- JWT access + refresh tokens; token blacklisting on logout

Product Catalogue

- Category-based browsing with slug routing
- Full-text product search
- Product detail: images, colours, sizes, specifications, materials
- Star rating & review count displayed per product

Shopping Cart

- Add items with selected colour and size
- Guest cart via session ID; merges on login
- Auto-calculated subtotals and totals
- Slide-over cart sidebar for quick access

Checkout & Payment

- Multi-field shipping address form
- Razorpay integration: UPI, cards, net banking, wallets
- Server-side payment signature verification
- Payment method details (card type, bank, UPI app) stored per order

Order Management

- Order number generation; full order history
- Status lifecycle: Pending → Processing → Shipped → Delivered → Cancelled
- Refund request, tracking, admin notes, and completion timestamp

Reviews

- Authenticated users rate products 1–5 stars
- One review per user per product (unique constraint)
- Aggregate rating auto-updated on the product

Section 07

Database Models

Model	Key Fields
User	email (unique), phone, address, is_google_user
Category	name, slug, description
Product	name, slug, price, stock, in_stock, rating, reviews_count
ProductImage	image / image_url, is_primary, order, color_name
ProductColor	product (FK), color_name
ProductSize	product (FK), size_name
ProductSpecification	product (FK), specification, order
ProductMaterial	shell, lining, care_instructions (JSON)
Cart	user (OneToOne) OR session_id (guest), total_price (property)
CartItem	cart, product, quantity, selected_color, selected_size
Order	user, order_number, total_amount, status, payment_status, Razorpay IDs
OrderItem	order, product_name, product_price, quantity, color, size
Review	product, user, rating (1–5), comment

Section 08

User Workflow

Step	Action	Detail
1	Visit Site	User lands on the homepage with hero banner, featured categories, and top deals
2	Browse / Search	Browse by category or use the search bar to find clothing products
3	View Product	Open product detail modal — images, colour picker, size selector, reviews
4	Add to Cart	Select colour + size → add to cart (works as guest too)
5	Login / Register	Create account with email or sign in with Google
6	Checkout	Fill shipping address (name, phone, city, state, PIN, country)
7	Payment	Pay via Razorpay — UPI, card, net banking, or wallet
8	Order Confirmed	Order number generated; confirmation email sent
9	Track Order	Monitor status in 'My Orders' (Pending → Delivered)
10	Leave Review	Rate the product and write a review after purchase

Section 09

Payment Integration — Razorpay

Razorpay is integrated as the primary payment gateway, supporting all major Indian payment methods. Payment verification is performed **server-side** using HMAC-SHA256 signature validation to prevent tampering.

Payment Method	Details Captured
UPI	VPA address; app inferred (e.g. GPay, PhonePe, Paytm)
Credit / Debit Card	Card type (credit/debit), network (Visa, Mastercard, RuPay)
Net Banking	Bank name stored (e.g. SBI, HDFC, ICICI)
Digital Wallet	Wallet name (e.g. Paytm, Amazon Pay)

Payment Lifecycle:

- Frontend calls backend to create a Razorpay order
- Razorpay checkout modal opens in the browser
- On success, payment ID + signature sent to backend for verification
- Backend verifies signature; order status updated to PAID
- On failure, order status remains PENDING or is set to FAILED

Section 10

Security Measures

Concern	Mitigation
Authentication	JWT access tokens (short-lived) + refresh tokens (long-lived); token blacklist on logout
Social Login	Google ID token verified server-side via google-auth library
Password Reset	Secure email link with UID + Django token; 24-hour expiry
Payment Integrity	Razorpay signature verified server-side (HMAC-SHA256)
CORS	django-cors-headers — only the frontend origin is whitelisted
Environment Secrets	python-dotenv — all API keys in .env, never in source code
Protected Routes	React PrivateRoute component redirects unauthenticated users
Stock Integrity	in_stock auto-derived from stock field; MinValueValidator on cart quantity
Duplicate Reviews	unique_together constraint on (product, user) in Review model

Section 11

Challenges & Solutions

Challenge	Solution
Guest vs. logged-in cart	Cart linked to user OR session_id; merged on login
Duplicate cart entries	unique_together on (cart, product, color, size)
Razorpay payment verification	Server-side HMAC signature check before marking PAID
Auto stock management	in_stock boolean auto-derived from stock quantity in model.save()
Google login conflicts	get_or_create pattern; is_google_user flag prevents password login
CORS in development vs production	Environment-aware ALLOWED_ORIGINS via django-cors-headers
Static files in production	WhiteNoise middleware serves static files without extra CDN setup
Password reset security	Django's default_token_generator with UID + expiring token

Section 12

Deployment Architecture

Component	Platform / Tool	Notes
Frontend	Vercel	Auto-deploy from Git; vercel.json handles SPA routing
Backend	Gunicorn WSGI	Production-grade Python server; runtime.txt pins Python version
Database	PostgreSQL	Managed DB; connection via dj-database-url env var
Static Files	WhiteNoise	Served inline with Django; no separate static CDN required
Media Images	Pillow + ImageField	Uploaded to products/ directory; also supports external URLs
Configuration	python-dotenv (.env)	All secrets outside source code
Python Version	runtime.txt	Pins exact Python version for reproducible builds

Section 13

Future Scope

Feature	Description
Admin Dashboard	GUI for product/order management; sales analytics and charts
Wishlist	Save products for later; shareable wishlists
Coupon & Discounts	Percentage and flat discount codes with expiry dates
SMS Notifications	Order status updates via Twilio or MSG91
Recommendations	AI-powered 'You May Also Like' based on browsing history
Multi-Vendor Support	Allow multiple sellers with separate dashboards
Mobile App	React Native app sharing the same Django REST backend
Size Recommendation	AI/ML model to suggest the right size based on measurements

■■ Section 14

Conclusion

WhatYouWear is a production-ready, full-stack e-commerce platform that brings together modern frontend and backend technologies to deliver a real-world shopping experience.

- Clean separation of concerns via a decoupled REST API architecture
- Secure, dual-mode authentication (JWT + Google OAuth 2.0)
- Full Indian payment gateway integration (Razorpay — UPI, cards, wallets)
- Scalable relational data models covering the full e-commerce domain
- Modern, animated UI with React 19, Tailwind CSS 4, and Framer Motion
- Independent, cloud-native deployment on Vercel and Gunicorn

This project covers the **complete software development lifecycle** — from requirements analysis and system design through implementation, security hardening, and cloud deployment — making it a comprehensive demonstration of full-stack web engineering skills.

Section 15

References & Tools

Tool / Framework	URL
React 19	https://react.dev
Vite 7	https://vitejs.dev
Tailwind CSS 4	https://tailwindcss.com
Framer Motion	https://www.framer.com/motion
React Router DOM v7	https://reactrouter.com
Django	https://www.djangoproject.com
Django REST Framework	https://www.django-rest-framework.org
SimpleJWT	https://django-rest-framework-simplejwt.readthedocs.io
Razorpay	https://razorpay.com/docs
Google OAuth 2.0	https://developers.google.com/identity
PostgreSQL	https://www.postgresql.org
Vercel	https://vercel.com
Gunicorn	https://gunicorn.org
WhiteNoise	http://whitenoise.evans.io